TZ受信機 性能評価スクリプト 使い方マニュアル

対象: TZ-tools ブランチ port-ros-measurements ／ 版 0.1（2026-10-21）／ 実機での動作は未確認

1. このマニュアルの範囲
2. 用語
3. 準備
4. 共通の使い方
5. 各スクリプト
6. 解析（rx_performance_analysis.py）
7. 疑似機器での試運転（`--dry-run`）
8. 設定ファイル device_config_ros.toml
9. 注意点と未確定事項
10. 改訂履歴

1. このマニュアルの範囲

TZ受信機の性能（受信機雑音温度 Trx とイメージ抑圧比 IRR）を測るスクリプト群の使い方を説明する。 ROSで動いていた旧システムのスクリプトを、ROSを使わない新システム（tz_controller）の上に移植したもの。 観測データの解析には使わない。

旧スクリプト（ROS）	新スクリプト	何をするか
yfactor_chopper_directon.py	yfactor_chopper.py	チョッパを回したまま、IF 4系統の Trx を測る
yfactor_sa_direction.py	yfactor_hotcold.py	HOT と COLD を手で入れ替えて、IF 4系統の Trx を測る
yfactor_with_search_sis_vol_direction-ver2.py 同 -ver2_auto.py	trx_map.py trx_map.py `--auto`	SISバイアス2つを格子状に振り、1系統の Trx マップを作る
yfactor_with_search_loatt_level_direction-ver2.py	loatt_search.py	LO減衰器の電流を振り、HL と VL の Trx を測る

irr_measurement2.py	irr_measure.py	1組のSISバイアスで、両偏波の IRR と Trx を測る
解析用 Jupyter notebook 各種	rx_performance_analysis.py	上の測定結果から Trx、IRR、マップを計算する

2. 用語

呼び方	中身
旧システム	ROSで動いていた制御一式（2019-2025）
新システム	tz_controller（r-yamada1998/TZ-controller）と TZ-tools。ROSなしで FAC 上で動く
制御PC	hinotoritz。Anapico SG（1st LO）用のコードは ~/nk にある
FAC	hinotoritz-fac。DAボードとADボードが刺さっている。新システムは ~/tz_workspace で動く
DAボード	CPZ-3346。SISバイアスを出す。DA ch5 から ch8 を使う
ADボード	CPZ-3177。SISのモニタを読む。AD ch9 から ch16 を使う
素子名	v1, h1, h2, v2。SISミクサ4本の名前（8章の対応表を参照）
IF系統	HU, HL, VU, VL（偏波 H/V と サイドバンド U/L）
Bias1, Bias2	trx_map.py で振る2つのバイアス。Bias1 は v1 と h1、Bias2 は v2 と h2 を同時に動かす（旧システムと同じ）

3. 準備

- FAC の ~/tz_workspace に、TZ-tools の次のファイルを置く: yfactor_chopper.py, yfactor_hotcold.py, trx_map.py, loatt_search.py, irr_measure.py, rx_performance_common.py, rx_performance_analysis.py, device_config_ros.toml
- tz_controller が入った環境で動かす（`uv run python ...`）。必要なパッケージ: numpy、ogameasure、pyinterface、matplotlib（図を作る場合）
- 本番の ~/tz_workspace/device_config.toml と、インストール済みの tz_controller は読まないし書き換えない
- 測定ごとの前提（チョッパ、LO、黒体）は5章の各スクリプトの項を見る

4. 共通の使い方

実行の形

旧システムと同じく、スクリプト名の後に引数を並べる。旧システムで対話入力していた値の多くは、引数で渡す形にした。

```
cd ~/tz_workspace
uv run python trx_map.py HL --bias1 6.0 8.0 21 --bias2 6.0 8.0 21
```

どのスクリプトも `-h` を付けると引数の一覧が出る。

共通オプション

オプション	意味
`--dry-run`	疑似機器で動かす。受信機には何も送らない（7章）
`--yes`	確認の [y/n] にすべて y と答える。`--dry-run` の試運転用
`--keep-sa`	スペアナの設定を変えない（手で設定したまま測る）
`--settle 秒`	切り替え後の待ち時間。既定値は旧システムと同じ

出力

実行した場所に、測定1回ごとにフォルダができる。

```
trx_map_261021_143005/
  meta.json      測定条件（引数、素子名とDA端子の対応、換算係数、スペアナ設定など）
  traces.csv     1行がスペアナの1トレース。先頭の列が条件（バンド、バイアスなど）、
                 続く p000, p001, ... がトレース（dBm）
  result.csv     解析結果（測定の最後に自動で作られる）
  trx_map.png    図（マップ系のみ）
```

旧システムの necstdb ではなく CSV なので、Excel や pandas でそのまま開ける。

安全のための確認

- 起動時に設定ファイルの換算係数を確かめ、旧システムの値（1/(3*0.94)）から1%以上ずれていれば止まる
- どのSISバイアスも 0 から 9.0 mV の外なら、出力する前に止まる（範囲は設定ファイルで変更可）

5. 各スクリプト

5.1 yfactor_chopper.py：チョッパでの Trx（4系統）

前提: チョッパが回っている。SISバイアスと LO は事前に設定しておく（このスクリプトは触らない）。

```
uv run python yfactor_chopper.py {filter,nofilter} FREQ
```

引数	意味
filter / nofilter	周波数分離フィルタの有無（記録用）
FREQ	1st LO SG の周波数（記録用。例 15.609）

流れ: y を待つ → HU, HL, VU, VL の順に IFスイッチを切り替え、10秒待って、ゼロスパンのトレースを1本取る。

結果: 系統ごとの Trx。

5.2 yfactor_hotcold.py：HOT と COLD を手で入れ替える Trx（4系統）

前提: SISバイアスと LO は事前に設定しておく。

```
uv run python yfactor_hotcold.py {filter,nofilter} [--freq FREQ]
```

流れ: HOT を置いて y → 4系統それぞれ、切り替え、10秒待ち、y で1本取る → COLD を置いて同じことをする。`--freq` を省くと周波数を聞かれる。

5.3 trx_map.py：Trx マップ（1系統）

前提: チョッパが回っている。LO が入っている。

```
uv run python trx_map.py {HU,HL,VU,VL} --bias1 START STOP NUM --bias2 START STOP NUM
[--auto]
```

引数	意味
HU など	測る IF系統
`--bias1` START STOP NUM	v1 と h1 に同じ値を入れて振る。START から STOP まで NUM 点
`--bias2` START STOP NUM	v2 と h2 に同じ値を入れて振る
`--auto`	Bias1 を1段変えるごとの y 確認をしない（旧 ver2_auto）。付けなければ旧 ver2 と同じく毎段 y を待つ

`--settle`	Bias2 を変えた後の待ち時間。既定 5.8 秒

流れ: Bias1 の各値で v1, h1 を設定 →（`--auto` でなければ y を待つ）→ Bias2 の各値で v2, h2 を設定、5.8秒待ってトレースを1本取る。

> 終了時に4本のSISバイアスをすべて 0 にする（旧システムと同じ）。測定後は運用値を設定し直すこと（initialize.py など）。

所要時間の目安: 21 × 21 点で約45分（待ち時間 5.8 秒の場合）。起動時に目安を表示する。

結果: trx_map.png。各マスの色と数字は同じ値から描いている（9章参照）。

5.4 loatt_search.py：LO減衰器の電流と Trx

前提: チョッパが回っている。LO が入っている。

```
uv run python loatt_search.py START STOP NUM FREQ [--bias V1 H1 H2 V2] [--traces N]
```

引数	意味
START STOP NUM	減衰器の電流 [mA] を START から STOP まで NUM 点。旧スクリプトでは3つ目を step と呼んでいたが、実際は点数だった
FREQ	1st LO SG の周波数（記録用）
`--bias`	測定中に保つSISバイアス。省くと設定ファイルの tuned 値（旧システムでは対話入力）
`--traces`	1点あたりのトレース数（既定 1）

流れ: バイアスを設定して y → 各電流で H を設定、60秒待って V を設定 → HL と VL を測る。最後に、残す H と V の電流を聞かれ、その値に設定する。

5.5 irr_measure.py：IRR（両偏波）

前提: LO が入っている。HOT と COLD は手で置く。試験信号は Phase Matrix FSW-0020 から出す（スクリプトが制御する）。

```
uv run python irr_measure.py V1 H1 H2 V2 [--usb-freq 16.609] [--lsb-freq 14.609]
```

引数	意味
V1 H1 H2 V2	SISバイアス [mV]。旧 set_sis_vol_direction.py と同じ順
`--usb-freq`, `--lsb-freq`	試験信号 SG の周波数 [GHz]。省くと聞かれる。SG の出力は6逓倍されて受信機に入る

流れ: バイアス設定、試験信号オフ → COLD、HOT の順に4系統 → USB 試験信号、LSB 試験信号の順に4系統。各系統で切り替え、10秒待ち、y で1本取る。

結果: 偏波ごとの IRR（USB, LSB）と Trx。バイアスは設定したまま終わる。

6. 解析（rx_performance_analysis.py）

測定スクリプトは最後に自動で解析する。解析だけをやり直すときは次のように実行する。

```
python rx_performance_analysis.py trx_map_261021_143005
python rx_performance_analysis.py irr-map 出力フォルダ irr_measure_A irr_measure_B ...
```

`irr-map` は、バイアスを変えて取った複数回の irr_measure をまとめ、偏波ごとに IRR と Trx のマップを作る（旧 irr_map_multi_data_integration.ipynb）。

計算	方法（旧 notebook と同じ）
チョッパの Trx	ゼロスパンのトレースから立ち上がりと立ち下がりを除き、平均より上を HOT、下を COLD として Y を求める
HOT/COLD の Trx	HOT と COLD のトレースの差（dB）の平均から Y を求める
Y から Trx	Trx = (Thot − Tcold Y)/(Y − 1)、Thot = 300 K、Tcold = 77 K
IRR	HOT/COLD の出力と、試験信号のピーク（信号 − HOT が最大の点）から計算

旧データでの照合: 2025-09-10 の Trx マップは旧図の色と1マスずつ一致、2025-09-18 の IRR は旧 notebook の値と小数第2位まで一致。

7. 疑似機器での試運転（`--dry-run`）

受信機につながずに、スクリプトが最後まで動くかを確かめるための機能。

- DAボード、スペアナ、IFスイッチ、LO減衰器、試験信号 SG の代わりに、それらしい形のデータを返す「疑似機器」を使う
- 受信機には何も送らないので、手元の PC でも FAC でも安全に動かせる
- 出てくる Trx や IRR の数値には物理的な意味はない。確かめられるのは、手順、記録、解析、作図が通ること
- 引数の打ち間違いや、所要時間の目安を本番前に確かめるのにも使える

```
python trx_map.py HL --bias1 6 8 5 --bias2 6 8 5 --dry-run --yes
```

8. 設定ファイル device_config_ros.toml

旧システムの動作を再現するための設定。旧システムのソースと 2025年9月の運用ログで確認した値にしてある。

項目	値	根拠
DA出力の換算	指令値 × 1/(3*0.94)	旧システムの設定と運用ログ。この値でミクサにかかる電圧が指令値と1%以内で一致する
素子名とDA端子	v1 = DA ch7, h1 = DA ch6, h2 = DA ch5, v2 = DA ch8	旧システムが実際に出力していた対応（2020-05以降）
DAレンジ	10V	本番と同じ。出力中にレンジを変えると一瞬電圧が変わるため
tuned（運用値）	v1 6.4, h1 7.3, h2 7.3, v2 6.3 mV	2025-2026 シーズンの旧システムの運用値
LO減衰器	ポート4 = H = 23 mA、ポート5 = V = 23 mA	旧システムの launch と運用値
IFスイッチの経路	HU 105, HL 106, VU 107, VL 108	旧システムの switch_writer.conf

> 本番の device_config.toml とは素子名の対応と換算係数が違う。本番側（新システム）では v1 = DA ch5, h2 = DA ch7、換算は × 0.94/3。同じ「v1 = 6.4」でも、どちらの設定で動かすかによってかかる電圧と端子が変わる。

9. 注意点と未確定事項

- **実機では未試験。**初回は `--dry-run` で手順を確かめてから、短い格子で試すこと
- **素子名と物理的な偏波の対応は未確定。**旧システムの書き込み側の名前と、読み取り側（新システムと同じ）の名前で、DA ch5 と ch7 の扱いが食い違っている。FAC で LO を入れ、DA端子を1本ずつ動かしてどの IF が変わるかを見れば決まる。決まったら設定ファイルを直す
- **IFスイッチ:** tz_controller の IFSwitch.close() は close ではなく open を送る。このスクリプト群は close を直接呼んで回避している
- **IRR のスペアナ設定:** 旧システムではスペアナを手で設定していた。既定値（中心 6 GHz、スパン 0.5 MHz）は旧 notebook の前提に合わせたもの。RBW などは meta.json に記録されるので確認すること
- **旧システムの数字入りマップ:** Trx マップ（trx_map-num 図）は数字が転置されていた（色が正しい）。IRR マップは色が1列ずれることがあった（数字が正しい）。新しい図では色と数字は同じ値から描いている

10. 改訂履歴

版	日付	内容
0.1	2026-10-21	初版。旧システムの6本を5本に移植、解析を追加

原稿は TZ-tools の docs/manual/rx_performance_manual.html。コードを変えたらここを直し、docs/manual/build_pdf.sh で PDF を作り直す。